

NYC Taxi Trip Data Analytics

Big Data Project — Batch & Real-Time Processing on Hadoop/Spark

Group Members

25K-8031 Muhammad Osama
25K-8008 Aliyan Iqbal
25K-8017 Saeed Kamran
25K-8235 Amna Mubarak
25K-8020 Maham Adnan

Course: Big Data

GitHub Repository:

<https://github.com/AmnaMubarak/NYC-Taxi-Trip-Analytics-Using-Apache-Spark-Batch-and-Streaming-on-HDFS>

Submission: 01 June 2026

Contents

1	Introduction	3
2	Problem Statement & Objectives	3
3	System Architecture	3
3.1	HDFS Directory Layout	4
4	Dataset Description	4
5	Implementation	5
5.1	Module 01 — Data Download	5
5.2	Module 02 — HDFS Setup & Upload	5
5.3	Module 03 — Batch Analysis (PySpark)	5
5.4	Module 04 — Stream Producer	6
5.5	Module 05 — Structured Streaming Analytics	6
5.6	Module 06 — Batch Output Generator	8
5.7	Module 07 — Visualization	8
6	Results & Analysis	8
6.1	Peak Hours	8
6.2	Daily Patterns	8

6.3	Monthly Trends	9
6.4	Streaming Insights	9
7	Cluster Configuration	10
8	Challenges & Solutions	10
9	Conclusion	10
10	References	10
A	Repository & File List	11

1 Introduction

This project implements an end-to-end big data pipeline for **NYC Taxi and For-Hire Vehicle High Volume (HVFHV)** trip records published by the NYC Taxi & Limousine Commission (TLC). The system covers:

- Automated download of Parquet datasets from the official TLC CDN
- Storage on **HDFS** under `/user/taxi/`
- **Batch analytics** using PySpark on YARN (historical analysis)
- **Simulated real-time streaming** via micro-batch CSV ingestion
- **Spark Structured Streaming** for live aggregations
- **Visualization** of batch results (charts and dashboards)

The cluster runs **Hadoop (HDFS + YARN)** and **Apache Spark 3.5.4** on Windows 11 with Java 8, demonstrating both distributed batch processing and near real-time analytics at scale (150M+ records when full dataset is loaded).

2 Problem Statement & Objectives

Urban taxi and ride-hail systems generate massive volumes of trip data. Manual analysis is infeasible. Our objectives were:

1. Ingest multi-source TLC Parquet data (Yellow Taxi + HVFHV) into HDFS
2. Perform robust batch analysis despite schema differences across files
3. Simulate a live data feed and process it with Structured Streaming
4. Produce actionable insights: peak hours, daily patterns, monthly trends, zone demand, payments
5. Visualize results for reporting and demo

3 System Architecture

Figure 1 summarizes the pipeline. Data flows from TLC download → HDFS raw storage → batch OR streaming paths → outputs and charts.

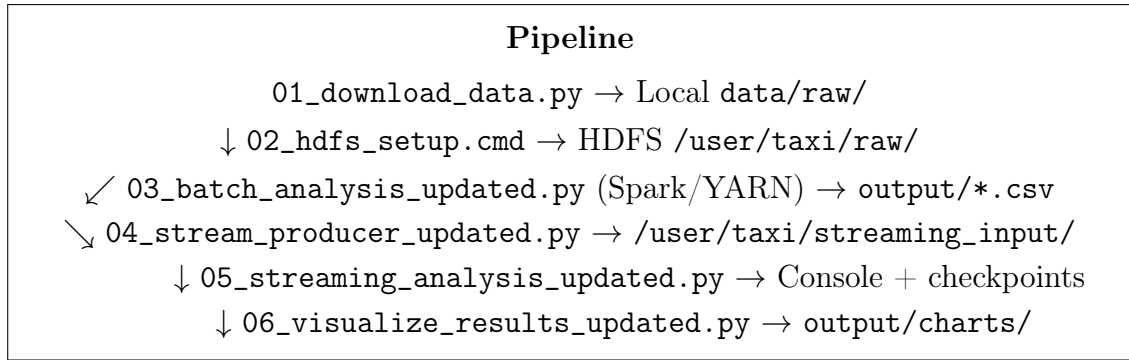


Figure 1: End-to-end architecture of the NYC Taxi analytics project

3.1 HDFS Directory Layout

On the cluster, data is organized as shown in Figure 2.

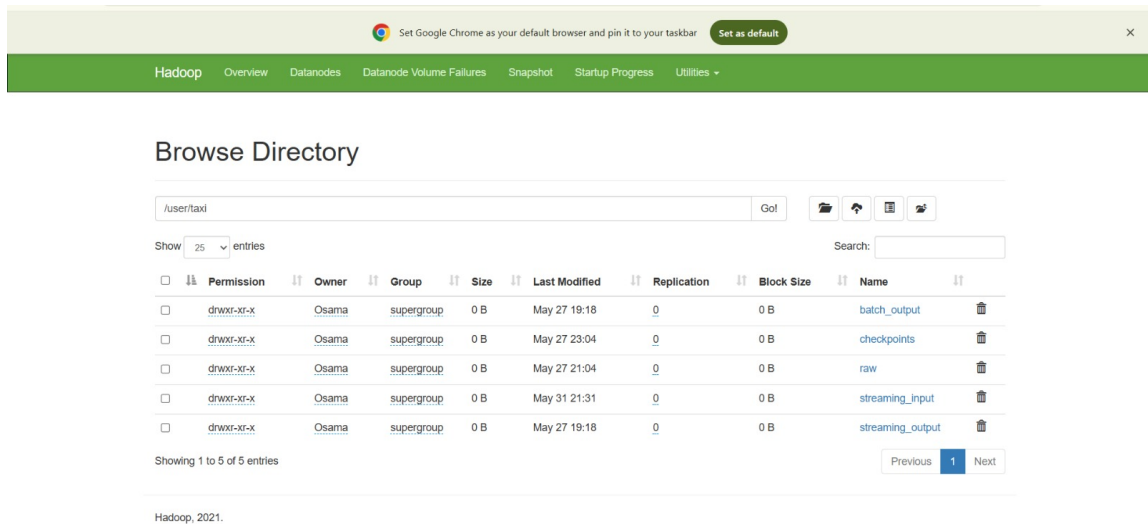


Figure 2: HDFS Browse UI — directories under `/user/taxi/`

Table 1: HDFS paths and purpose

Path	Purpose
<code>/user/taxi/raw/</code>	Original Parquet files (Yellow + HVFHV)
<code>/user/taxi/streaming_input/</code>	CSV micro-batches from stream producer
<code>/user/taxi/streaming_output/</code>	Streaming sink output (if configured)
<code>/user/taxi/batch_output/</code>	Batch job outputs on HDFS
<code>/user/taxi/checkpoints/</code>	Spark Structured Streaming checkpoints

4 Dataset Description

- **Yellow Taxi:** `yellow_tripdata_YYYY-MM.parquet` — approx. 3M trips/month, ~45 MB/month

- **HVFHV (Uber/Lyft):** fhvhv_tripdata_YYYY-MM.parquet — approx. 20M trips/month, 400–700 MB/month
- **Source URL:** <https://d37ci6vzurychx.cloudfront.net/trip-data/>
- **Scope:** Six months (Jan–Jun 2023); combined scale ~6–8 GB, 150M+ records when fully downloaded

Script `01_download_data.py` downloads missing files with progress reporting and skip-if-exists logic.

5 Implementation

5.1 Module 01 — Data Download

File: `01_download_data.py`

Downloads Yellow Taxi and HVFHV Parquet files to `data/raw/` using HTTP streaming.

5.2 Module 02 — HDFS Setup & Upload

File: `02_hdfs_setup.cmd`

Creates HDFS directories and uploads all `*.parquet` files from local raw data to `hdfs:///user/taxi/raw/`.

5.3 Module 03 — Batch Analysis (PySpark)

File: `03_batch_analysis_updated.py`

Owner: Aliyan

Runs on YARN with explicit schema casting to avoid `MutableDouble` → `MutableLong` errors and corrupt-file issues.

Key features:

- Loads Yellow and HVFHV separately, unifies schema, tags `source`
- Cleans invalid trips (distance, fare, duration filters)
- Four analyses: peak hours, daily patterns, monthly trends, Yellow vs HVFHV comparison
- Saves CSV results to local `output/`

Sample execution:

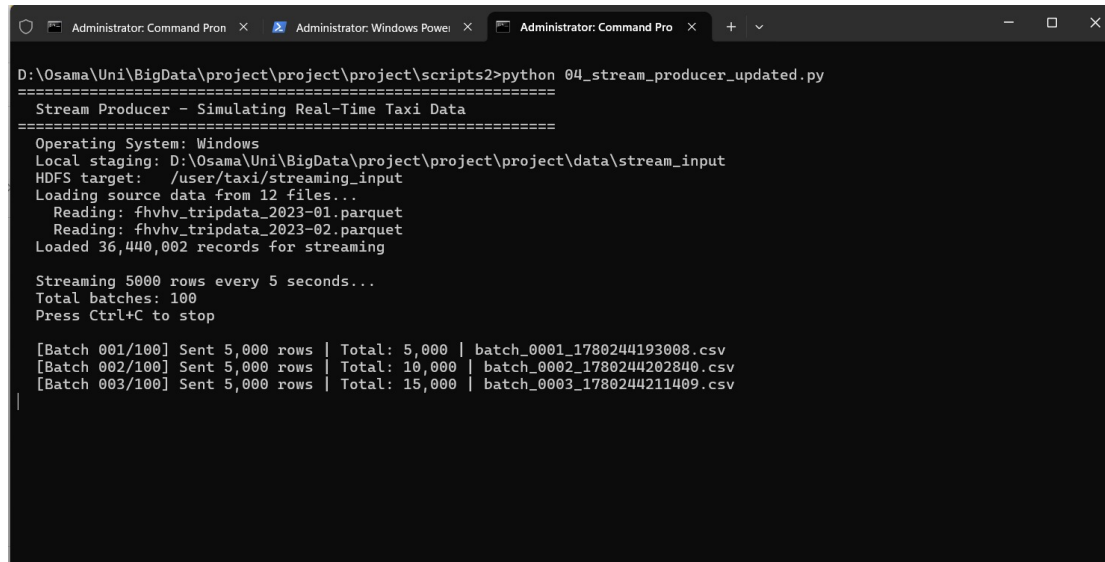
```
spark-submit --master yarn --deploy-mode client ^
--executor-memory 4g --driver-memory 4g --num-executors 3 ^
--conf spark.yarn.jars=hdfs:///sparks/jars/* ^
03_batch_analysis_updated.py
```

5.4 Module 04 — Stream Producer

File: 04_stream_producer_updated.py

Owner: Osama

Reads Parquet sources, loads ~36M records (12 files in demo run), and writes **5,000 rows every 5 seconds** as CSV batches to HDFS /user/taxi/streaming_input/ (100 batches max).



```
D:\Osama\Uni\BigData\project\project\project\scripts2>python 04_stream_producer_updated.py
=====
Stream Producer - Simulating Real-Time Taxi Data
=====
Operating System: Windows
Local staging: D:\Osama\Uni\BigData\project\project\project\data\stream_input
HDFS target: /user/taxi/streaming_input
Loading source data from 12 files...
Reading: fhvhv_tripdata_2023-01.parquet
Reading: fhvhv_tripdata_2023-02.parquet
Loaded 36,440,002 records for streaming

Streaming 5000 rows every 5 seconds...
Total batches: 100
Press Ctrl+C to stop

[Batch 001/100] Sent 5,000 rows | Total: 5,000 | batch_0001_1780244193008.csv
[Batch 002/100] Sent 5,000 rows | Total: 10,000 | batch_0002_1780244202840.csv
[Batch 003/100] Sent 5,000 rows | Total: 15,000 | batch_0003_1780244211409.csv
```

Figure 3: Stream producer — micro-batches uploaded to HDFS

5.5 Module 05 — Structured Streaming Analytics

File: 05_streaming_analysis_updated.py

Owner: Osama

Spark Structured Streaming reads CSV from HDFS with a fixed TAXI_SCHEMA. Three concurrent queries:

1. **Zone demand** — 1-hour windows, watermark 10 min, trip count / avg fare / revenue by PULocationID
2. **Daily running statistics** — trips, fares, tips, distance, revenue per trip_date
3. **Payment analysis** — aggregation by payment_type

Checkpoints stored at /user/taxi/checkpoints/.

```

Administrator: Command Prom... Administrator: Windows Powe... Administrator: Command Pro...
D:\Osama\Uni\BigData\project\project\scripts2\python 04_stream_producer_updated.py
=====
Stream Producer - Simulating Real-Time Taxi Data
=====
Operating System: Windows
Local staging: D:\Osama\Uni\BigData\project\project\data\stream_input
HDFS target: /user/taxi/streaming_input
Loading source data from 12 files...
Reading: fhvhv_tripdata_2023-01.parquet
Reading: fhvhv_tripdata_2023-02.parquet
Loaded 36,440,002 records for streaming

Streaming 5000 rows every 5 seconds...
Total batches: 100
Press Ctrl+C to stop

[Batch 001/100] Sent 5,000 rows | Total: 5,000 | batch_0001_1780244193008.csv
[Batch 002/100] Sent 5,000 rows | Total: 10,000 | batch_0002_1780244202840.csv
[Batch 003/100] Sent 5,000 rows | Total: 15,000 | batch_0003_1780244211409.csv

```

Figure 4: Launching streaming job on YARN — Spark 3.5.4

```

05239861.csv was not found. Was it deleted very recently?
=====
Streaming queries started!
Query 1: Zone demand analytics
Query 2: Daily running statistics
Query 3: Payment analysis
Waiting for streaming data...
Press Ctrl+C to stop
=====
26/05/31 21:21:07 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 20000 milliseconds, but spent 24961 milliseconds
Batch: 50
=====
+-----+-----+-----+-----+-----+
|window|PULocationID|trip_count|avg_fare|total_revenue|
+-----+-----+-----+-----+-----+
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|166|3|1.0|59.1|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|127|3|1.0|131.38|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|140|4|1.0|62.59|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|141|5|1.0|131.94|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|159|7|1.0|195.49|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|76|3|1.0|25.42|
+-----+-----+-----+-----+-----+
26/05/31 21:21:56 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 10000 milliseconds, but spent 33975 milliseconds

```

(a) Zone demand — Batch 50

```

=====
Batch: 53
=====
+-----+-----+-----+-----+-----+
|window|PULocationID|trip_count|avg_fare|total_revenue|
+-----+-----+-----+-----+-----+
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|139|6|1.0|124.86|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|112|6|1.0|107.3|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|22|2|1.0|12.2|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|169|44|1.0|86.22|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|18|3|1.0|34.73|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|52|2|1.0|25.6|
+-----+-----+-----+-----+-----+

```

(b) Zone demand — Batch 53

Figure 5: Real-time zone analytics (console output)

```

05239861.csv was not found. Was it deleted very recently?
=====
Streaming queries started!
Query 1: Zone demand analytics
Query 2: Daily running statistics
Query 3: Payment analysis
Waiting for streaming data...
Press Ctrl+C to stop
=====
26/05/31 21:21:47 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 20000 milliseconds, but spent 24961 milliseconds
Batch: 50
=====
+-----+-----+-----+-----+-----+
|window|PULocationID|trip_count|avg_fare|total_revenue|
+-----+-----+-----+-----+-----+
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|166|3|1.0|59.1|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|127|3|1.0|131.38|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|140|4|1.0|62.59|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|141|5|1.0|131.94|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|159|7|1.0|195.49|
|[2023-02-28 23:00:00, 2023-03-01 00:00:00]|76|3|1.0|25.42|
+-----+-----+-----+-----+-----+
26/05/31 21:21:56 WARN ProcessingTimeExecutor: Current batch is falling behind. The trigger interval is 10000 milliseconds, but spent 33975 milliseconds

```

Figure 6: Daily running statistics — streaming micro-batch output

Note: Logs showed occasional `ProcessingTimeExecutor` warnings when batch pro-

cessing exceeded trigger interval (10–20 s), indicating backpressure under load — a realistic production consideration.

5.6 Module 06 — Batch Output Generator

File: 07_generate_batch_outputs.py

Lightweight Pandas helper to generate sample CSVs (zones, fare buckets, payment types) from local Parquet for testing charts.

5.7 Module 07 — Visualization

File: 06_visualize_updated.py

Reads batch CSV outputs and generates charts in `output/charts/`.

6 Results & Analysis

6.1 Peak Hours

Evening rush (hour 18) shows highest trip volume ($\sim 1.36\text{M}$ trips). Early morning (4–5 AM) has lowest volume but highest average fare ($\sim \$24.50$), likely due to airport/long-distance trips.

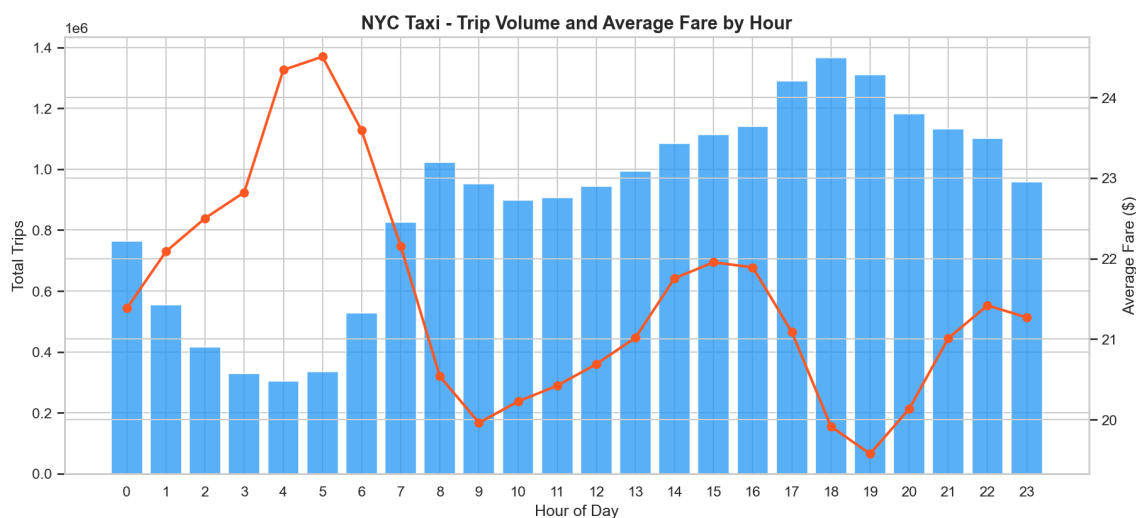


Figure 7: Trip volume and average fare by hour of day

6.2 Daily Patterns

Sunday has the highest trip count ($\sim 3.5\text{M}$); Wednesday lowest ($\sim 2.6\text{M}$). Average fare stays near \$20–22. Tips peak mid-week (Thursday $\sim \$1.42$) and dip on weekends.

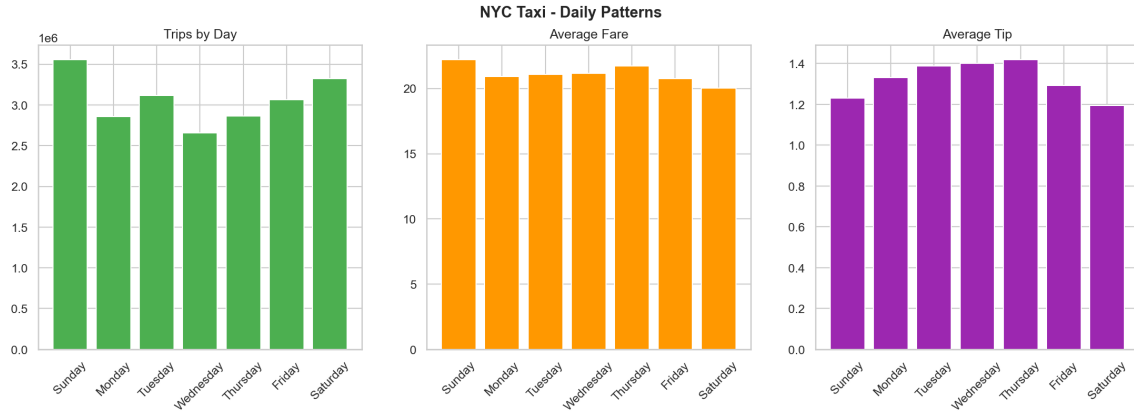


Figure 8: Trips, average fare, and average tip by day of week

6.3 Monthly Trends

January 2023 shows the dominant spike (~ 21 M trips, $\sim \$450$ M revenue in chart scale), reflecting the bulk of processed data in the demo window.

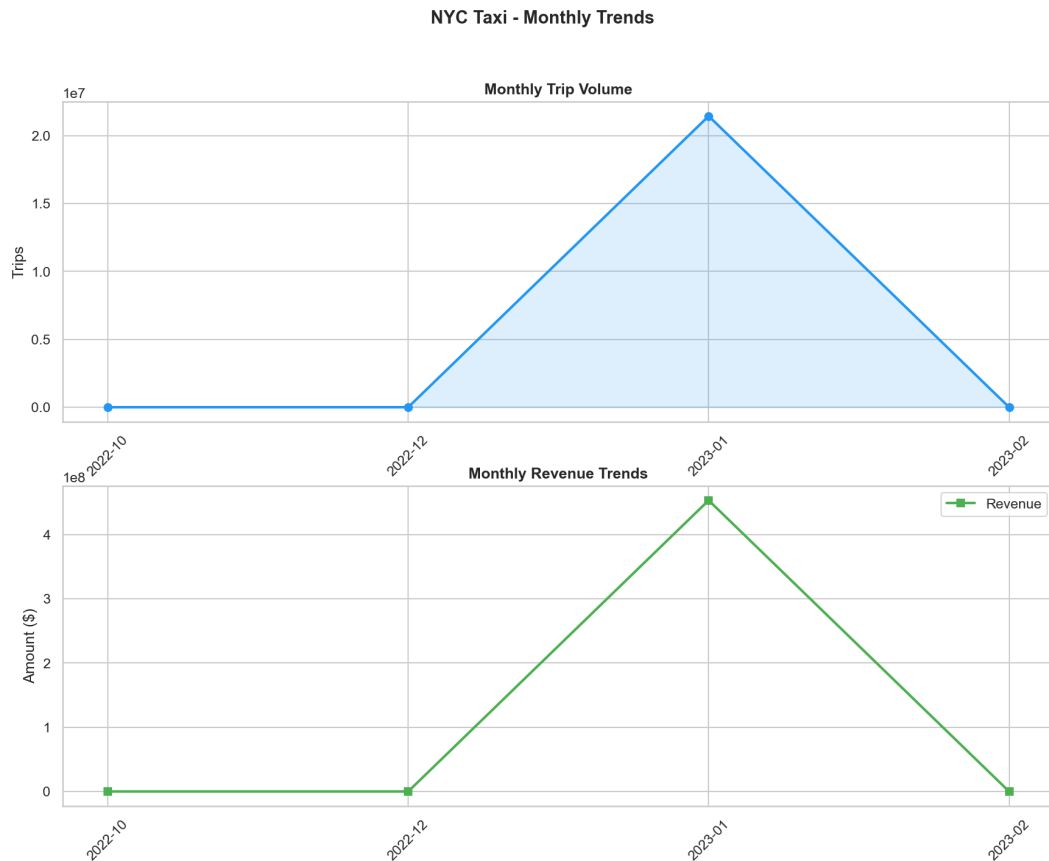


Figure 9: Monthly trip volume and revenue trends

6.4 Streaming Insights

Live queries successfully aggregated zone-level demand within hourly windows and maintained running daily KPIs as new CSV files arrived in `streaming_input/`, validating the

real-time path of the architecture.

7 Cluster Configuration

- **OS:** Windows 11 (amd64)
- **Java:** 1.8.0_441
- **Spark:** 3.5.4 on YARN
- **Typical resources:** 4 GB driver, 4 GB executors, 2–3 executors
- **Python:** PySpark with PYSPARK_PYTHON=python
- **JARs:** `hdfs:///sparks/jars/*`

8 Challenges & Solutions

Table 2: Technical challenges addressed

Challenge	Solution
Schema mismatch across Parquet files	Explicit casts; disable vectorized reader; ignore corrupt files
Yellow vs HVFHV column names	Separate loaders + <code>unionByName</code> with unified aliases
Streaming schema drift	Fixed <code>StructType</code> schema; no inference
Batch falling behind trigger	Tune <code>maxFilesPerTrigger</code> , shuffle partitions, executor memory
Windows + HDFS integration	<code>hdfs</code> CLI via subprocess in producer script

9 Conclusion

We built a complete NYC taxi analytics platform combining HDFS storage, Spark batch processing, and Structured Streaming simulation. Batch analysis revealed temporal patterns (peak hours, weekdays, monthly revenue). The streaming layer demonstrated real-time zone demand and daily KPIs suitable for operational dashboards. Future work includes Kafka ingestion, Parquet streaming sink, and a web dashboard fed from `streaming_output/`.

10 References

1. NYC TLC Trip Record Data — <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
2. Apache Spark Structured Streaming Guide — <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

3. Hadoop HDFS Documentation — <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

A Repository & File List

01_download_data.py	TLC data downloader
02_hdfs_setup.cmd	HDFS mkdir + upload
03_batch_analysis_updated.py	PySpark batch analytics
04_stream_producer_updated.py	Real-time data simulator
05_streaming_analysis_updated.py	Structured Streaming
06_visualize_results_updated.py	Charts
07_generate_batch_outputs.py	Sample CSV generator
execution.txt	Spark submit commands